

[연구보문]

정적분석을 위한 통합 언패킹 및 역난독화 자동화 시스템 개발

이진성¹, 이호웅², 김지훈³, 임성호³, 정도준³, 변준석^{3†}

¹호서대학교 대학원 컴퓨터공학과 대학원생

²호서대학교 대학원 컴퓨터공학과 교수

³국립과학수사연구원 디지털과 감정관

(접수: 2023년 4월 7일; 수정: 2023년 10월 16일; 게재확정: 2023년 10월 23일)

[Original Research Article]

Development of Integrated Unpacking and Deobfuscation Automation System
for Static Analysis

Jinseong Lee¹, Howoong Lee¹, Jihun Kim², Seongho Lim², Dojoon Jung², Junseok Byun^{2†}

¹Department of Computer Engineering, Hoseo Graduate School, Asan, Korea

²Digital Analysis Division, National Forensic Service, Wonju, Korea

(Received: 7 April 2023; Revised: 16 October 2023; Accepted: 23 October 2023)

Abstract Recently, technologies that make static analysis difficult are applied to most malicious codes, and the most representative technologies are packing and obfuscation technologies. Therefore, research on unpacking and deobfuscation techniques is essential for effective static analysis. Therefore, in this paper, an integrated unpacking and deobfuscation automation system was developed to solve this problem, and a 94.9% recovery rate was confirmed through a verification process.

Key Words: Malware, Static Analysis, Unpacking, Deobfuscation

† Corresponding author

Tel : +82-33-902-5323 Fax : +82-33-902-5943

E-mail : jsbyun77@korea.kr

I. 서론

악성코드(Malware)란 악성 소프트웨어(Malicious Software)의 합성어로 컴퓨터 또는 네트워크를 손상시키거나 파괴하는 비도덕적인 범죄 목적으로 컴퓨터, 네트워크 또는 데이터에 무단 액세스하도록 작성된 소프트웨어를 말한다[1].

과거의 바이러스나 웜과 같은 전통적인 악성코드들은 자가 복제하여 시스템 전반에 영향을 미치거나, 특정 기능을 수행하기 위해 설치된 후 시스템을 제어하였다. 이러한 악성코드들은 사용자의 인지 없이 무단으로 설치되며, 시스템 성능을 저하시키거나 중요 데이터를 손상시키는 등 시스템의 파괴 및 마비를 목적으로 하는 행위가 주를 이루었다. 그러나, 최근 정보통신기술의 급격한 발전과 관련 기술이 산업/사회에서 광범위하게 활용됨에 따라, 최근의 악성코드들은 시스템 파괴를 목적으로 하기보다는 개인정보 탈취, 금전적인 이득 등을 위해 제작되고 있다[2]. 이러한 악성코드들은 기술적인 해킹 방법보다는 정치 현안, 사회 이슈, 기업 정보 등 공격 대상이 관심 가질 만한 주제에 대하여 SNS·포털 등 다양한 경로를 통하여 정보를 수집하고 공격 자원으로 활용한다[3]. 최근에 제작되고 있는 대다수의 악성코드 또한 여전히 이전의 기술적인 방법들을 활용하거나 참조하고 있다. 그렇기 때문에 여러 안티-바이러스 프로그램이나 악성코드 분석가들은 여전히 과거부터 누적된 이전의 악성코드들의 공격기술, 형태에 대한 공통점, 패턴, 시그니처 등을 통해서 악성코드를 탐지, 분류하는 등에 활용하고 있다.

그러나, 최근의 악성코드들은 이전의 악성코드들과는 다르게 탐지 기능을 우회하거나 분석을 어렵게 하기 위해서, Anti-Virtual Machine, Anti-Debugging, Packing, 난독화 등과 같은 분석 방해 지연 기술이 탑재되어 있다[4]. 이 중 실행 파일 압축과 코드 난독화 기술은 모두 정적분석을 방해하는 대표적인 기술들이다. 고도화되고 있는 사이버 범죄에 사용된 악성코드 일부는 공격자가 실행 가능한 실행압축 형태로 배포되는 경우가 많으며, 이 기법을 Packing(이하 패킹)이라고 한다. 그리고 이러한 패킹 기능을 제공하는 도구를 Packer(이하 패커)라고 한다[5]. 또한, 패킹과 같이 정적분석을 방해하는 기법 중 하나인 코드 난독화는 프로그램 코드를 해석하기 어렵도록 변형시키는 것을 주목적으로 한다.

정적분석은 파일의 겉모습을 관찰하여 분석하는 방법으로 파일의 종류, 크기, PE 헤더 정보, Import/Export API, 내부 문자열, 실행압축 여부, 등록 정보, 디버깅 정보, 디지털 인증서 등의 다양한 내용을 확인하는 분석 방법이다[6]. 그러나, 코드와 내부 데이터를 압축하거나 인코딩을 적용하는 실행 파일 압축 기법이 적용되어있으면 정적분석을 위해 다시 디코딩하거나 압축을 해제하여야만 원 코드 확인이 가능하다. 그뿐만 아니라 실행 파일 압축은 코드 블록을 재배치하는 등 코드를 변형하여 원 코드의 함수 등을 해석하기 어렵게 하며 행위 파악을 위한 중요한 지표인 API 함수들을 확인하기 어렵게 한다.

또한, 코드 난독화는 변수나 함수의 이름을 무의미한 문자열이나 해석하기 어려운 문자열 등으로 변경한다. 그뿐만 아니라 코드의 흐름을 해석하기 어렵도록 더미 코드를 삽입하여 코드 해석 시 동작을 파악하기 복잡하고 어렵게 한다.

최근 악성코드의 수가 급증하고 있으며, 공격자는 방어 환경을 우회하고 탐지를 회피하기 위해 악성코드를 지속 고도화해 나가고 있다. 또한, 악성코드 자동화 제작 도구 등 악성코드를 쉽게 구매하거나 제작할 수 있다[7].

이에 대응하기 위해서 많은 분석가들도 악성코드 분석, 탐지에 자동화를 적용하고 있지만, 위에서 언급한 정적분석을 방해하는 기술로 인한 어려움을 겪고 있는 것이 현실이다.

이런 문제를 해결하고자 동적 분석 방법이 정적분석의 대안으로 제안되기도 하였으나, 이 방법 또한 Anti-Virtual Machine과 Anti-debugging 같은 분석 방해 기법으로 분석에 어려움이 있는 상황이다[4]. 특히, 최근 악성코드들은 Command & Control(이하 C&C) 서버와의 통신을 기반으로 동작을 하고 있기 때문에, C&C 서버가 동작하지 않는 경우, 동적 분석 자체가 불가능한 상황이 발생한다. 또한, 사용자의 특정 행위나 특정 시간에 동작하는 트리거형 악성코드에 있어서는 그 행위 분석이나 탐지가 되지 않을 수 있다. 이와 같은 문제를 해결하기 위해서는 정적분석을 통해 모든 코드를 분석하는 과정이 불가피하다.

정적분석은 악성 행위를 코드상에서 분석하기 때문에 심층적이고 정확한 분석이 가능하며, 효과적인 정적분석을 위해서는 위의 방해 기술들을 해결하는 것이 선행되어야 한다. 이를 위해, 악성코드에 적용된 패킹 기법을 탐지 및

식별하는 다양한 연구가 진행되었으며[8], 특정 패킹 기법을 상세하게 분석하고 분석된 결과를 바탕으로 원본 바이너리 파일을 복원하는 언패킹 연구도 진행되었다[9][10]. 그러나, 패킹 기법을 식별하고 자동으로 언패킹을 진행하는 자동화된 통합 도구 개발 연구는 아직 더딘 상황이다[11].

현재 RPA(Robotic Process Automation)를 사용하는 솔루션은 인건비 감소와 기업 내부 프로세스의 운영 우수성을 향상시키고 COVID-19로 인해 자동화가 점점 중요해지고 있다[12].

본 논문에서는 위에서 언급한 정적분석을 방해하는 기술들인 실행 파일 압축과 난독화를 효율적으로 해결하기 위해 자동화된 언패킹 및 역난독화 방법을 제안하고자 하며, 이를 통해 분석 시간, 비용적인 손실 등은 최소화하지만 악성 행위 분석은 더욱더 정확하고 심층적으로 진행될 수 있도록 하고자 한다.

논문의 구성은 제 II장에서는 본 연구를 통해 개발한 시스템과 환경에 대해 서술하고 III장에서는 개발한 시스템의 성능을 검증한 결과에 대해 서술하였다, IV장에서 결론으로 마무리 짓는다.

II. 시스템 및 환경

1. 시스템 구성 및 환경

본 연구를 위해 Fig. 1과 같이 VMware에 Windows OS를 기반으로 Anaconda를 활용하여 python 3.8버전의 conda 가상환경을 구축하였다. 구축된 가상환경에서 Visual studio Community 2022와 Pycharm Community를 사용하여 개발을 진행하였다.

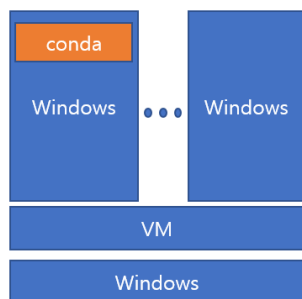


Fig. 1. 시스템 환경 구성.

또한, 통합 자동화 도구의 기본 프레임워크로 오픈소스인 peframe-ds[13]를 기반으로 개발을 진행하였다. 언패킹과 역난독화 도구로는 UPX[14], unlicense[15], VMPdump[16]를 기반으로 개발을 진행하였다.

최종적으로 패킹 및 난독화된 파일이 분석이 가능한 상태로 복원이 되었는지 확인을 위해, 정적분석 도구인 IDA Pro와 Ollydbg를 사용하여 정적분석이 가능한지 검증하고 PE구조를 확인할 수 있는 도구인 PEView를 사용하여 IAT(Import Address Table)가 복구되는지 검증을 진행하였다.

실험을 진행하기 위해 Visual Studio를 사용하여 테스트 프로그램을 작성하였으며, 최종적으로 성능 검증을 위해 2022년 3월에 배포된 DikeDataset[17]에서 확보한 악성코드를 사용하였다.

2. 실험 방법 및 분석

본 연구의 목적인 통합 자동화 언패킹 및 역난독화 분석 시스템 개발은 다음과 같은 과정을 거쳐 진행하였다. 첫 번째로 대상 파일을 정적분석하기 위해 python기반으로 한 오픈소스인 peframe-ds를 기반으로 소스 코드 개발을 진행하였다. peframe-ds에서 제공하는 기본적인 기능은 활용하면서, 본 연구를 위한 기능 추가를 위해 Fig. 2와 같이 필요한 모듈을 추가하였으며, 언패킹 및 역난독화를 위한 도구인 UPX, unlicense, VMPdump를 peframe-ds가 설치될 때 같이 설치가 되도록 패키지 개발을 진행하였다.

```

swig
pefile
pyreadline3
python-magic-bin
yara-python
virustotal-api
oletools
psutil
  
```

Fig. 2. 필요한 모듈.

두 번째로 Fig. 3, Fig. 4와 같이 언패킹 및 역난독화 도구를 선택할 때 대상 파일에서 사용된 패커를 식별하고, Section Name에 따라 UPX, Themida, VMProtect를 분류하여 각각 해당하는 도구가 동작할 수 있도록 개발하였다.

```
elif user_input == "tools":
    packer = result['pinfo']['features']['packer']
    detect_section = result['pinfo']['sections']['details']
    u_feature, t_feature, v_feature = False, False, False
    u_section, t_section, v_section = False, False, False

    f_packer, f_section = "", ""
    upx, themida, vmp = False, False, False

    for pack in packer:
        packer_name = pack
        # packer_name = pack
        if packer_name.find('upx') > -1 or packer_name.find('UPX') > -1:
            f_packer = "Fine Features"
            upx = True
        if packer_name.find('themida') > -1 or packer_name.find('Themida') > -1:
            f_packer = "Fine Features"
            themida = True
        if packer_name.find('VMProtect') > -1:
            f_packer = "Fine Features"
            vmp = True

    for section in detect_section:
        section_name = section['section_name']
        if section_name.find('upx') > -1 or section_name.find('UPX') > -1:
            f_section = "Fine Section"
            upx = True
        if section_name.find('themida') > -1 or section_name.find('Themida') > -1:
            f_section = "Fine Section"
            themida = True
        if section_name.find('vmp') > -1 or section_name.find('VMProtect') > -1 or section_name.find('VMProtect') > -1:
            f_section = "Fine Section"
            vmp = True
```

Fig. 3. 패커 식별 코드.

```
if upx:
    if f_section == "":
        print(f_packer[:-1], "!!!")
    elif f_packer == "":
        print(f_section, "!!!")
    else:
        print(f_packer+f_section, "!!!")
    ex_tools.exec_upx()
    break
elif themida:
    if f_section == "":
        print(f_packer[:-1], "!!!")
    elif f_packer == "":
        print(f_section, "!!!")
    else:
        print(f_packer+f_section, "!!!")
    os_type = result['filetype']
    if os_type.find('x86'):
        if os_type.find('64'):
            ex_tools.exec_themida('64')
        else:
            ex_tools.exec_themida('32')
    break
elif vmp:
    if f_section == "":
        print(f_packer[:-1], "!!!")
    elif f_packer == "":
        print(f_section, "!!!")
    else:
        print(f_packer+f_section, "!!!")
    ex_tools.exec_vmprotect()
    break
if upx and themida and vmp:
    print("Unpacking 및 역난독화를 진행할 수 없습니다.")
    break
```

Fig. 4. 예외 처리 코드.

세 번째로 패커를 식별에 성공한 후, 언패킹 및 역난독화를 진행하도록 개발하였다. 먼저, Fig. 5, Fig. 6, Fig. 7, Fig. 8, Fig. 9와 같이 사용자 이름과 conda 가상환경의 이름을 찾아 도구가 있는 경로에 분석 대상 파일을 이동시키고 도구를 이용하여 언패킹 및 역난독화를 진행하였다. 그리고 언패킹 및 역난독화로 생성된 결과 파일을 사용자의 바탕화면 경로에 이동시키도록 개발하였다.

```
import os
import getpass
import shutil
import psutil
from npa import npaccli

def get_env_name():
    os.system("conda info > export_env_info.txt")
    env_file = open("./export_env_info.txt")
    data_file = env_file.readlines()
    for line in data_file:
        if 'active environment' in line:
            print("Find environment name")
            env_name = line.split()
            env_name = env_name[-1]
            env_file.close()
            os.system("del export_env_info.txt")
            return env_name
    print("not find enviroment name")
    return None
```

Fig. 5. conde 가상환경 이름 식별 코드.

```
def path_paser(origin, user, env_name, tools_name):
    filename = npaccli.filename
    filename = origin
    tools_path = "C:/Users/" + user + "/anaconda3/envs/" + env_name + "/Lib/site-packages/NPA-0.1.0-py3.8.egg/npa/Tools/"
    os.chdir(tools_path)

    origin_path = filename.split("\\")
    origin_file = origin_path[-1]
    origin_path = filename.strip(origin_file)
    # 파일 형식 확인(헤더값을 or 실행코드를 or DLL)
    category = origin_file.rfind('.')
    if category > 0:
        if tools_name == "upx":
            out_file = tools_path + origin_file[:category] + ".reversers" + origin_file[category:]
        elif tools_name == "themida":
            out_file = tools_path + 'unpacked.' + origin_file[:category] + origin_file[category:]
        elif tools_name == "vmp":
            out_file = tools_path + 'unpacked.' + origin_file[:category] + origin_file[category:]
    else:
        if tools_name == "upx":
            out_file = tools_path + origin_file + ".reversers"
        elif tools_name == "themida":
            out_file = tools_path + 'unpacked.' + origin_file
        elif tools_name == "vmp":
            out_file = tools_path + 'unpacked.' + origin_file

    out_path = out_file.split("/")
    out_file = out_path[-1]

    return category, origin_path, origin_file, tools_path, out_file
```

Fig. 6. 분석파일을 도구가 있는 경로에 이동시키는 코드.

```

def file_export(origin_path, origin_file, out_path, out_file):
    shutil.move(os.path.join(out_path, out_file), os.path.join(origin_path, out_file))
    print("export file path :", origin_path + out_file)
    os.system("cd %Userprofile%")

def exec_upx():
    user = getpass.getuser()
    env_name = get_env_name()
    if env_name != None:
        category, origin_path, origin_file, tools_path, out_file = path_paser(npaccli.filename, user, env_name, tools_name='upx')
        #find dot
        if category > 0:
            upx = 'upx -d --strip-relocs=0 -o ' + tools_path+out_file + ' ' + origin_path+origin_file
        else:
            upx = 'upx -d --strip-relocs=0 -o ' + tools_path+out_file + ' ' + origin_path+origin_file

        os.system(upx)
        file_export(origin_path, origin_file, tools_path, out_file)
    else:
        print("please conda env activate [%env_name%]")

```

Fig. 7. 언패킹 및 결과파일 이동 코드.

```

def exec_themida(os_type):
    user = getpass.getuser()
    env_name = get_env_name()
    if env_name != None:
        category, origin_path, origin_file, tools_path, out_file = path_paser(npaccli.filename, user, env_name, tools_name='themida')
        #find dot
        if os_type == '32':
            themida = "unlicense "
        elif os_type == '64':
            themida = "unlicense_64 "

        if category > 0:
            themida += origin_path+origin_file
        else:
            themida += origin_path+origin_file

        print(themida)
        os.system(themida)
        file_export(origin_path, origin_file, tools_path, out_file)
    else:
        print("please conda env activate [%env_name%]")

```

Fig. 8. Themida 언패킹 및 역난독화 코드.

```

def exec_vmprotect():
    user = getpass.getuser()
    env_name = get_env_name()
    category, origin_path, origin_file, tools_path, out_file = path_paser(npaccli.filename, user, env_name, tools_name='vmp')
    pid = None
    pars_start = "start /b "+origin_path + origin_file
    # os.system(pars_start)
    os.system(pars_start)
    for proc in psutil.process_iter():
        if origin_file in proc.name():
            pid = proc.pid
            break
        else:
            continue

    tools_path = "C:/Users/" + user + "/anaconda3/envs/" + env_name + "/Lib/site-packages/WPA-0.1.0-py3.8.egg/npa/Tools/"
    os.chdir(tools_path)
    # pid = str(input('분석 파일의 pid를 입력해 주세요! == '))
    ep = npaccli.result['peinfo']['entrypoint']
    vmp = 'VMPDump.exe ' + str(pid) + ' ' + " " + "-ep="+str(ep)

    os.system(vmp)

    for proc in psutil.process_iter():
        if origin_file in proc.name():
            kill_parse = "taskkill /F /pid " + str(pid)
            os.system(kill_parse)
            break

```

Fig. 9. VMProtect 언패킹 및 역난독화 코드.

이 과정에서 자동화를 위해 가장 중요한 부분이 패커를 정확하게 식별하는 것이라고 할 수 있다. 패커 식별이 정확하지 않을 경우, 적절하지 않은 언패킹 및 역난독화 도구를

선택하여 오류를 발생할 수 있기 때문이다. 따라서, 본 연구에서는 패커를 보다 정확하게 식별할 수 있도록 총 49개의 yara rule을 Table 1과 같이 개발하여 적용하였다.

Table 1. Yara rule

<pre> rule UPX_Gratuito: PEiD { strings: \$a = {60 BE ?? ?0 4? 00 8D BE ?? ?? F? FF} condition: \$a at pe.entry_point } </pre>
<pre> rule UPX_Shut_v01: PEiD { strings: \$a = {55 8B EC 83 C4 D8 60 E8 00 00 00 00 5A 81 EA ?? ?? ?? ?? 8B DA C7 45 D8 00 00 00 00 8B 45 D8 40 89 45 D8 81 7D D8 80 00 00 00 74 0F 8B 45 08 89 83 ?? ?? ?? ?? FF 45 08 43 EB E1 89 45 DC 61 8B 45 DC C9 C2 04 00 55 8B EC 81 C4 7C FF FF FF 60 E8 00 00 00 00} condition: \$a at pe.entry_point } </pre>
<pre> rule UPX_DLL: PEiD { strings: \$a = {48 89 4C 24 08 48 89 54 24 10 4C 89 44 24 18 80 FA 01 0F 85 ?? 02 00 00 53 56 57 55 48 8D 35 DD ?? ?? FF 48 8D BE 00 ?? ?? FF 57 31 DB 31 C9 48 83 CD} condition: \$a at pe.entry_point } </pre>
<pre> rule UPX_exe: PEiD { strings: \$a = {53 56 57 55 48 8D 35 ?? ?? ?? FF 48 8D BE ?? ?? ?? FF} condition: \$a at pe.entry_point } </pre>
<pre> rule UPX_DLL_file: PEiD { strings: \$a = {48 89 4C 24 08 48 89 54 24 10 4C 89 44 24 18 80 FA 01 0F 85 ?? 0B 00 00 53 56 57 55 48 8D 35 ?? ?? ?? FF 48 8D BE 00} condition: \$a at pe.entry_point } </pre>
<pre> rule UPX_ARM_Little: PEiD { strings: \$a = {FF 4F 2D E9 20 30 8F E2 07 00 B3 E8 10 0C 93 E8 02 90 A0 E1 0D 00 00 EB 04 00 A0 E3 01 00 00 EB FF 4F BD E8 20 F0 9F E5 14 F0 9F E5 00 ?? ?? 00} \$b = {FF 4F 2D E9 20 30 8F E2 07 00 B3 E8 10 0C 93 E8 02 90 A0 E1 0D 00 00 EB 04 00 A0 E3 01 00 00 EB FF 4F BD E8 20 F0 9F E5 14 F0 9F E5 00} condition: \$a at pe.entry_point or \$b at pe.entry_point } </pre>

```

rule UPX: PEiD
{
    strings:
        $a = {60 BE ?? 04 ?0 08 DB E? ?? ?F ?F F?}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_v0_81: PEiD
{
    strings:
        $a = {B9 00 00 BE 00 00 89 F7 1E A9 B5 80 8C C8 05 05 00 8E D8 05 00 00 8E C0 FD F3 A5 FC
2E 80 6C 12 10 73 E7 92 AF AD 0E 0E 0E 06 1F 07 16 BD 00 00 BB 00 80 55 CB 55 50 58 21 0A 03 03 07}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_v1_25_Markus: PEiD
{
    strings:
        $a = {31 2E 32 35 00 55 50 58}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_v2_93: PEiD
{
    strings:
        $a = {60 BE ?? ?? ?? ?? 8D BE ?? ?? ?? ?? 57 89 E5 8D 9C 24 ?? ?? ?? ?? 31 C0 50 39 DC 75
FB 46 46 53 68 ?? ?? ?? ?? 57 83 C3 04 53 68 ?? ?? ?? ?? 56 83 C3 04 53 50 C7 03 03 00 02 00}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_Modified: PEiD
{
    strings:
        $a = {60 BE ?? ?? ?? ?? 8D BE ?? ?? ?? ?? 57 EB ?? 90 8A 06 46 88 07 47 01 DB 75 ?? 8B 1E
83 EE FC 11 DB 72 ?? B8 ?? 00 00 00 01 DB 75 ?? 8B 1E 83 EE FC 11 DB 11 C0 01 DB 73 ?? 75 ?? 8B 1E}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_exe_NRV2E: PEiD
{
    strings:
        $a = {60 BE 00 ?? ?? ?? 8D BE 00 ?? ?? ?? 57 83 CD FF EB 10 90 90 90 90 90 90 8A 06 46 88
07 47 01 DB 75 07 8B 1E 83 EE FC 11 DB 72 ED B8 01 00 00 00 01 DB}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_v0_80: PEiD
{
    strings:
        $a = {8A 06 46 88 07 47 01 DB 75 07 8B 1E 83 EE FC 11 DB 72 ED B8 01 ?? ?? ?? 01 DB 75 07
8B 1E 83 EE FC 11 DB 11 C0 01 DB 73 ?? 75 ?? 8B 1E 83 EE FC}
    condition:
        $a at pe.entry_point
}

```

```
rule UPX_v1_25_Stub: PEiD
```

```
{
    strings:
        $a = {60 BE 00 ?? ?? 00 8D BE 00 ?? ?? FF}
    condition:
        $a at pe.entry_point
}
```

```
rule UPX_v2_02: PEiD
```

```
{
    strings:
        $a = {E9 ?? ?? ?? ?? 8D A4 24 00 00 00 00 8D 49 00 55 8B EC 83 C4 F8 60 C6 45 FF 00 C7 45
F8 00 00 00 00 8B 7D 08 8B 75 0C 8B 55 10 8B 5D 1C 33 C9 EB ?? 8B C1 03 C3 3B 45 20 77 ?? 51 56 57 52 8B CB
85 C9 74}
    condition:
        $a at pe.entry_point
}
```

```
rule UPX_v3_0: PEiD
```

```
{
    strings:
        $a = {60 BE ?? ?? ?? 00 8D BE ?? ?? ?? FF 57}
    condition:
        $a at pe.entry_point
}
```

```
rule UPX_v3_93_LZMA: PEiD
```

```
{
    strings:
        $a = {E8 ?? ?? ?? ?? 55 53 51 52 48 01 FE 56 41 ?? ?? ?? 0F 85 ?? ?? ?? ?? 55 48 89 E5 44 8B
09 49 89 D0 48 89 F2 48 8D 77 02 56 8A 07 FF CA 88 C1 24 07}
        $b = {E8 ?? ?? ?? ?? EB 0E 5A 58 59 97 60 8A 54 24 20 E9 11 0B 00 00 60 8B 74 24 24 8B 7C
24 2C 83 CD FF 89 E5 8B 55 28 AC 4A 88 C1 24 07 C0 E9 03 BB 00 FD FF FF D3 E3 8D A4 5C 90 F1 FF FF 83 E4
E0 6A 00 6A}
    condition:
        $a at pe.entry_point or $b at pe.entry_point
}
```

```
rule UPX_v3_93_NRV2x: PEiD
```

```
{
    strings:
        $a = {E8 ?? ?? ?? ?? 55 53 51 52 48 01 FE 56 48 89 FE 48 89 D7 31 DB 31 C9 48 83 CD FF E8
?? ?? ?? ?? 01 DB 74 ?? F3 C3 8B 1E 48 83 EE FC 11 DB 8A 16 F3}
    condition:
        $a at pe.entry_point
}
```

```
rule UPX_v3_95_LZMA: PEiD
```

```
{
    strings:
        $a = {50 52 E8 ?? ?? ?? ?? 55 53 51 52 48 01 FE 56 41 ?? ?? ?? 0F 85 ?? ?? ?? ?? 55 48 89 E5
44 8B 09 49 89 D0 48 89 F2 48 8D 77 02 56 8A 07 FF CA 88 C1}
    condition:
        $a at pe.entry_point
}
```

```

rule UPX_v3_95_NRV2x: PEiD
{
    strings:
        $a = {50 52 E8 ?? ?? ?? ?? 55 53 51 52 48 01 FE 56 48 89 FE 48 89 D7 31 DB 31 C9 48 83 CD
FF E8 ?? ?? ?? ?? 01 DB 74 ?? F3 C3 8B 1E 48 83 EE FC 11 DB 8A}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_v3_95: PEiD
{
    strings:
        $a = {53 56 57 55 48 8D 35 ?? ?? ?? FF 48 8D BE ?? ?? ?? FF 57 31 DB 31 C9 48 83 CD FF E8
50 00 00 00 01 DB 74 02 F3 C3 8B 1E 48 83 EE FC 11 DB 8A 16 F3 C3}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_v3_9_x: PEiD
{
    strings:
        $a = {E8 ?? ?? ?? ?? 55 53 51 52 48 01 FE 56 41 80 F8 0E 0F 85 6C 0A 00 00 55 48 89 E5 44 8B
09 49 89 D0 48 89 F2 48 8D 77 02 56 8A 07 FF CA 88 C1}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_v3_9_X_DLL: PEiD
{
    strings:
        $a = {80 7C 24 08 01 0F 85 ?? ?? 00 00 60 BE ?? ?? ?? ?? 8D BE ?? ?? ?? FF}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_HiT_v0_0_1: PEiD
{
    strings:
        $a = {94 BC ?? ?? ?? 00 B9 ?? 00 00 00 80 34 0C ?? E2 FA 94}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_ShIt_v0_1: PEiD
{
    strings:
        $a = {E8 00 00 00 00 5E 83 C6 14 AD 89 C7 AD 89 C1 AD 30 07 47 E2 FB AD FF E0 C3 00 ?? ??
00 ?? ?? ?? 00 ?? ?? ?? 01 ?? ?? 00 55 50 58 2D 53 68 69 74 20 76 30 2E 31 20 2D 20 77 77 77 2E 62 6C 61 63 6B
6C 6F 67 69 63 2E 6E 65 74 20 2D 20 63 6F 64 65 20 62 79 20 5B 35 30 30 6D 68 7A 5D}
    condition:
        $a at pe.entry_point
}

```

```

rule UPX_ARM_Cpu: PEiD
{
    strings:
        $a = {01 00 51 E3 ?? 00 00 1A FF 4F 2D E9 ?? 30 8F E2 ?? ?? ?? E8 ?? ?? ?? ?? ?? ?? ?? ??
?? ?? ?? ?? ?? 00 ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? 14}
    condition:
        $a at pe.entry_point
}

```

```

rule Themida_WinLicense_v1_8: PEiD
{
    strings:
        $a = {B8 ?? ?? ?? ?? 60 0B C0 74 58 E8 00 00 00 00 58 05 ?? ?? ?? ?? 80 38 E9 75 03 61 EB 35
E8 00 00 00 00 58 25 00 F0 FF FF 33 FF 66 BB ?? ?? 66 83 ?? ?? 66 39 18 75 12 0F B7 50 3C 03 D0 BB ?? ?? ??
?? 83 C3 ?? 39 1A 74 07 2D 00 10 00 00 EB DA 8B F8 B8}
        condition:
            $a at pe.entry_point
}

```

```

rule Themida_WinLicese_v1_Oreans: PEiD
{
    strings:
        $a = {00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ?? ?? ?? ?? ?? ?? ?? ?? 00 00 00 00 00
00 00 00 00 00 00 ?? ?? ?? ?? ?? ?? ?? ?? 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 4B 45
52 4E 45 4C 33 32 2E 64 6C 6C 00 00 00 43 72 65 61 74 65 46 69 6C 65 41 00 00 00 45 78 69 74 50 72 6F 63 65 73
73 00 43 4F 4D 43 54 4C 33 32 2E 64 6C 6C 00 00 00 49 6E 69 74 43 6F 6D 6D 6F 6E 43 6F 6E 74 72 6F 6C 73 00
00 00 00 00 00}
        condition:
            $a at pe.entry_point
}

```

```

rule Thmida_WinLicese_v1_NoCompression: PEiD
{
    strings:
        $a = {8B C5 8B D4 60 E8 00 00 00 00 5D 81 ED ?? ?? ?? ?? 89 95 ?? ?? ?? ?? 89 B5 ?? ?? ??
?? 89 85 ?? ?? ?? ?? 83 BD ?? ?? ?? ?? ?? 74 0C 8B E8 8B E2 B8 01 00 00 00 C2 0C 00 8B 44 24 24 89 85 ?? ??
?? ?? 6A 45 E8 A3 00 00 00 68 9A 74 83 07 E8 DF 00 00 00 68 25}
        $b = {8B C5 8B D4 60 E8 00 00 00 00 5D 81 ED ?? ?? ?? ?? 89 95 ?? ?? ?? ?? 89 B5 ?? ?? ??
?? 89 85 ?? ?? ?? ?? 83 BD ?? ?? ?? ?? ?? 74 0C 8B E8 8B E2 B8 01 00 00 00 C2 0C 00 8B 44 24 24 89 85 ?? ??
?? ?? 6A 45 E8 A3 00 00 00 68 9A 74 83 07 E8 DF 00 00 00 68 25 4B 89 0A E8 D5 00 00 00 E9 ?? ?? ?? ?? 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00}
        condition:
            $a at pe.entry_point or $b at pe.entry_point
}

```

```

rule Themida_Unkown_Version: PEiD
{
    strings:
        $a = {52 ?? ?? ?? ?? ?? EB ?? ?? ?? ?? ?? ?? EB ?? ?? ?? ?? ?? 49 ?? ?? ?? ?? 52 54 54 FF ??
?? ?? ?? ?? 5A 4A ?? ?? ?? ?? 5A E9 ?? ?? ?? ?? 00 ?? 00 ?? ?? ?? ?? ?? ?? ?? 00 ?? 00 ?? ?? ?? ?? ?? 00 ?? 60
00 ?? ?? ?? ?? 00 ?? 00 ?? 00 ?? ?? ?? ?? ?? E8}
        condition:
            $a at pe.entry_point
}

```

```

rule Themida_Winlicense_v2_1_1_0: PEiD
{
    strings:
        $a = {83 EC 04 50 53 E8 01 00 00 00 CC 58 8B D8 40 2D 00 ?? ?? 00 2D 2B E8 5F 00 05 20 E8
5F 00 80 3B CC 75 19 C6 03 00 BB 00 10 00 00 68 ?? ?? ?? ?? 68}
        condition:
            $a at pe.entry_point
}

```

```

rule Themida_WinLicense_v1_9_x_x: PEiD
{
    strings:
        $a = {B8 00 00 00 00 60 0B C0 74 ?? E8 00 00 00 00 58 05 ?? 00 00 00 80 38 E9 75 ?? 61 EB}
    condition:
        $a at pe.entry_point
}

```

```

rule Themida_WinLicese_v2_2_7_0: PEiD
{
    strings:
        $a = {68 00 00 00 00 68 01 00 00 00 68 00 00 40 00 68 00 ?? ?? 00 E9 00 04 00 00 04 22 00 00
00 00 00 00 C6 21 00 00 00 00 00 00 A2 21 00 00 00 00 00 00 48}
    condition:
        $a at pe.entry_point
}

```

```

rule Themida_WinLicese_v3_0_x_0: PEiD
{
    strings:
        $a = {E8 82 01 00 00 41 52 49 89 E2 41 52 49 8B 72 10 49 8B 7A 20 FC B2 80 8A 06 48 FF C6
88 07 48 FF C7 BB 02 00 00 00 00 D2 75 07 8A 16 48 FF C6 10 D2 73}
    condition:
        $a at pe.entry_point
}

```

```

rule Themida_Winlicense_v3_0_x: PEiD
{
    strings:
        $a = {E8 4B 01 00 00 53 89 E3 53 8B 73 08 8B 7B 10 FC B2 80 8A 06 46 88 07 47 BB 02 00 00
00 00 D2 75 05 8A 16 46 10 D2 73 EA 00 D2 75 05 8A 16 46 10 D2 73}
    condition:
        $a at pe.entry_point
}

```

```

rule VMProtect_v1_70_4: PEiD
{
    strings:
        $a = {68 ?? ?? ?? ?? E8 ?? ?? ?? ?? FA ?? ?? ?? ?? ?? ?? ?? 54 FA ?? E9 ?? ?? ?? ?? 6E ?? ??
?? AD ?? ?? ?? ?? ?? ?? CC 5B 53 AB ?? ?? ?? ?? D9 ?? ?? 41 ?? ?? ?? D6 6D ?? ?? A4 48 ?? ?? ?? ?? ?? ??
?? ?? ?? ?? ?? 5C ?? ?? 48 FC D6 ?? ?? 48 FB 6D D9 ?? ?? ?? ?? E8}
    condition:
        $a at pe.entry_point
}

```

```

rule VMProtect_v1_25_0: PEiD
{
    strings:
        $a = {68 ?? ?? ?? ?? E8 ?? ?? ?? ?? 66 0F B7 06}
        $b = {68 ?? ?? ?? ?? E8 ?? ?? ?? ?? 66 8B 06}
    condition:
        $a at pe.entry_point or $b at pe.entry_point
}

```

```

rule VMProtect_v1_25_1: PEiD
{
    strings:
        $a = {68 ?? ?? ?? ?? E8 ?? ?? ?? ?? 89 EC 58}
        $b = {68 ?? ?? ?? ?? E8 ?? ?? ?? ?? 89 EC 59}
    condition:
        $a at pe.entry_point or $b at pe.entry_point
}

```

```
#include "windows.h"
#include "tchar.h"

int _tmain(int argc, TCHAR* argv[])
{
    MessageBox(NULL,
        L"Packer : Themida",
        L"test code",
        MB_OK);

    return 0;
}
```

Fig. 11. Themida 테스트 코드.

```
#include "windows.h"
#include "tchar.h"

int _tmain(int argc, TCHAR* argv[])
{
    MessageBox(NULL,
        L"Packer : VMProtect",
        L"test code",
        MB_OK);

    return 0;
}
```

Fig. 12. VMProtect 테스트 코드.

3. 분석기기 및 조건

분석환경을 구성하기 위해 VMware Workstation Pro 16버전에 Windwos 10 Pro 21H2버전을 사용하였다, 또한 anconda 가상환경은 conda 22.9.0버전에 python 3.8버전을 사용하였다.

파일을 정적분석하기 위해 python 3.6.6버전 이상을 지원하는 peframe-ds 6.1.0버전을 사용하였다.

통합 자동화 언패킹 및 역난독화 분석 시스템에서 지원하는 패커 및 난독화 도구의 종류와 버전은 Table 2과 같다.

단, UPX의 경우 정상적인 UPX 소스를 변경하여 패킹된 파일의 경우는 제한적으로 동작할 수 있다.

Table 2. 패커 및 난독화 도구 지원 버전

도구명	지원 버전
UPX	UPX 3.x
Themida	Themida 2.x, 3.x
VMProtect	VMProtect 3.x

III. 결과 및 고찰

윈도우즈 운영체제의 실행 파일 형식인 PE(Portable Executable, 이하 PE) 구조를 패킹 및 난독화를 수행하고, 이를 본연구를 통해 개발된 통합 자동화 언패킹 및 역난독화 분석 시스템을 통해 자동으로 PE구조를 정적분석하여 패커 및 난독화 도구 식별 후 언패킹 및 역난독화를 진행하여 원본 PE구조 또는 분석 가능한 형태의 PE구조로 복원되는지 검증하였다. 또한, 복원된 PE구조의 IAT가 정상적으로 복구되는 것을 확인하였다.

먼저, UPX 언패킹 결과는 Fig. 13과 같이 원본 PE구조의 형태로 복구된 것을 확인하였다. Fig. 14와 같이 정상 PE구조를 가지는 테스트 코드를 UPX를 사용하여 패킹을 진행하면, Fig. 15와 같이 section header부분과 section name이 UPX로 변하면서 패킹이 된다. 이후 패킹 된 파일을 Fig. 16과 같이 패킹된 상태에서의 PE구조에 대한 기본적인 정보 분석이 가능하다. 또한, Fig. 17과 같이 언패킹 명령을 수행하면 자동으로 언패킹이 진행된 것을 확인할 수 있다.

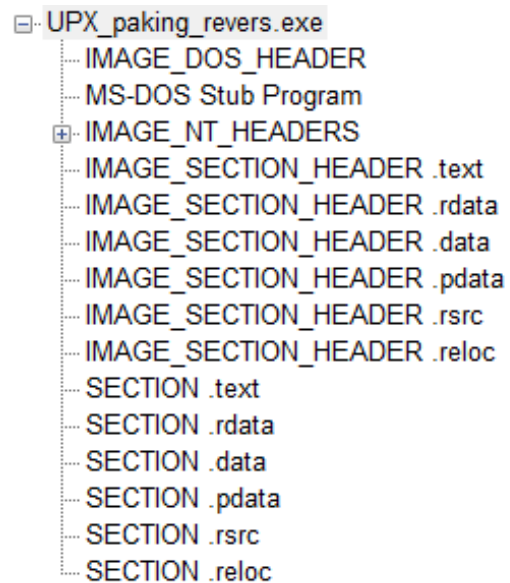


Fig. 13. UPX 언패킹 결과파일 PE구조.

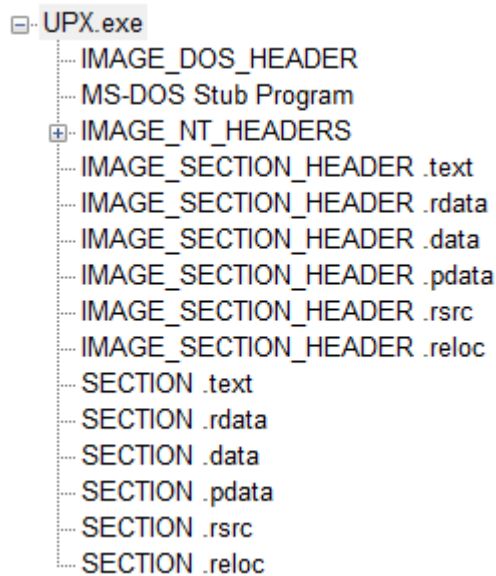


Fig. 14. UPX 원본 PE구조.

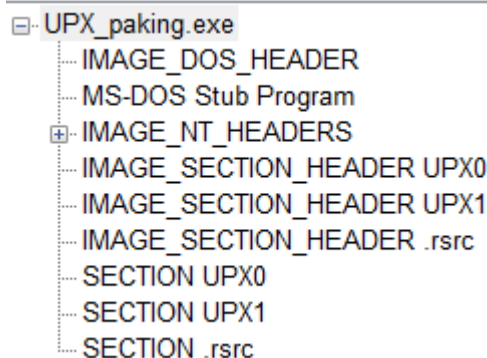


Fig. 15. UPX로 패키징된 파일 PE구조.

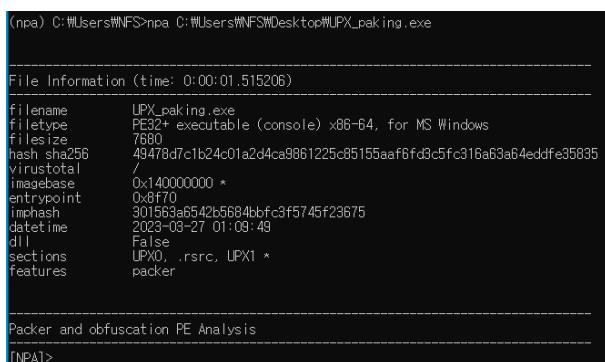


Fig. 16. 패킹파일 분석 시작.

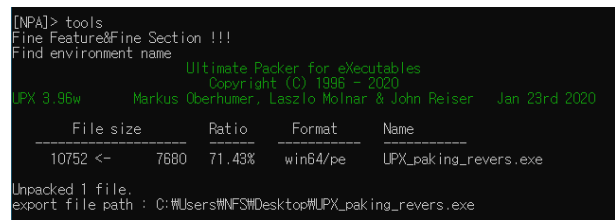


Fig. 17. UPX 언패킹.

두 번째, Themida로 패킹된 파일에 대한 언패킹 결과는 Fig. 18과 같이 원본 PE구조와 비교했을 때, themida로 패킹이 되면서 themida에서 자체적으로 생성된 section 부분이 삭제되지 않았지만, 원본 PE구조에 있는 section 부분은 언패킹된 것을 확인하였다. Fig. 19와 같이 정상 PE구조를 가지는 테스트 코드를 Themida 를 사용하여 패킹이 진행하면, Fig. 20과 같이 name이 없는 section이 추가되고 section name이 themida로 되어 패킹 및 난독화가 된 파일을 얻었다. 이후, 패킹된 파일을 Fig. 21과 같이 패킹된 상태에서의 PE구조에 대한 기본적인 정보 분석이 가능한 것을 확인할 수 있다. 또한, Fig. 22와 같이 언패킹 명령을 수행하면 자동으로 언패킹 및 역난독화가 진행된 것을 확인할 수 있다.

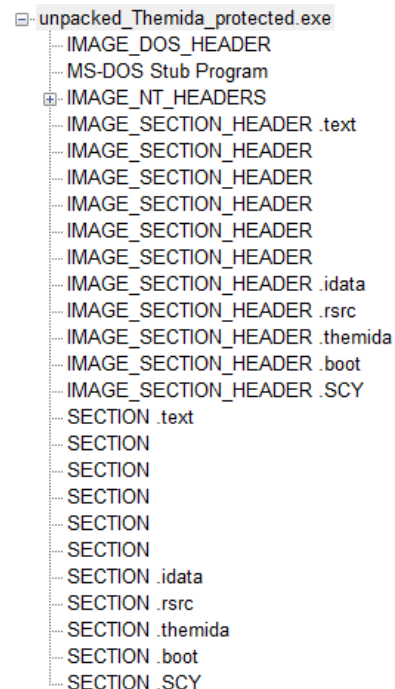


Fig. 18. Themida 언패킹 된 PE구조.

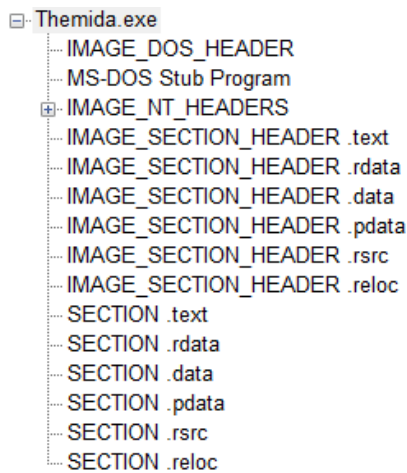


Fig. 19. Themida 원본 PE구조.

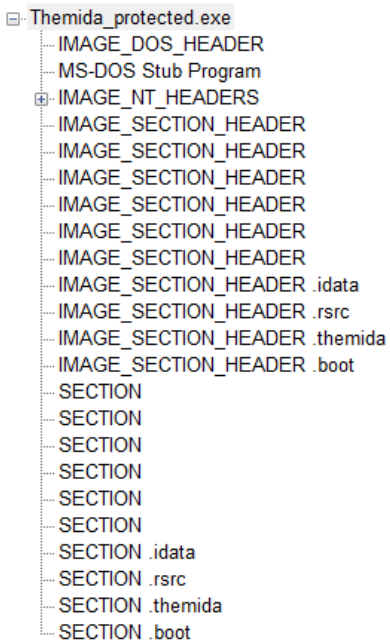


Fig. 20. Themida로 패킹된 PE구조.

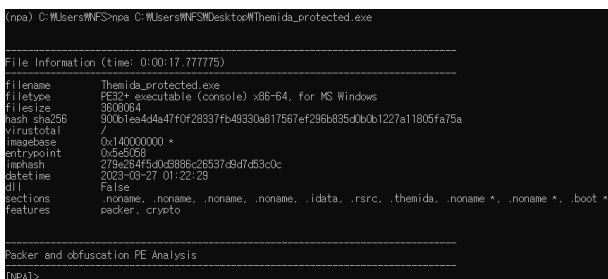


Fig. 21. 패킹파일 분석 시작.

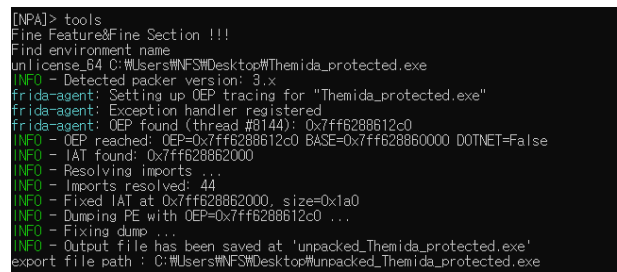


Fig. 22. Themida 언패킹.

마지막으로, VMProtect 언패킹 결과는 Fig. 23과 같이 원본 PE구조로 복구되지 않았지만, 원본 PE구조가 가지고 있는 section이 복구되어 정적분석이 가능한 것을 확인하였다. Fig. 24와 같이 정상 PE구조를 가지는 테스트 코드를 VMProtect를 사용하여 패킹을 진행하면, Fig. 25와 같이 section header의 개수가 증가하고 section의 개수가 줄어들어 패킹 및 난독화가 되었다. 이후 패킹된 파일을 Fig. 26과 같이 패킹된 상태에서의 PE구조에 대한 기본적인 정보를 보여준다. 또한, Fig. 27, Fig. 28과 같이 언패킹 명령을 수행하면 자동으로 언패킹 및 역난독화가 진행된 것을 확인할 수 있다.

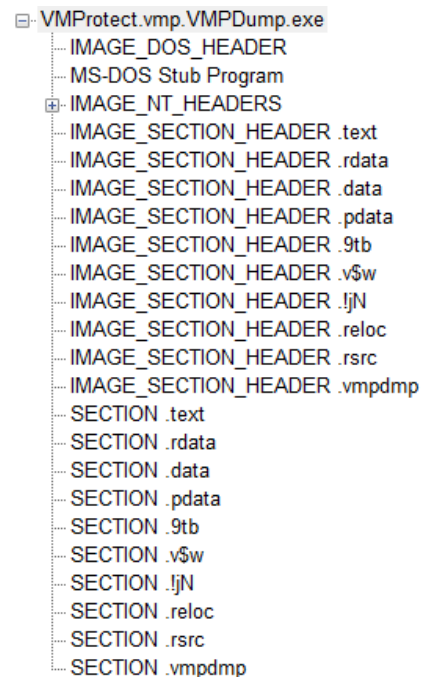


Fig. 23. VMProtect 언패킹된 PE구조.

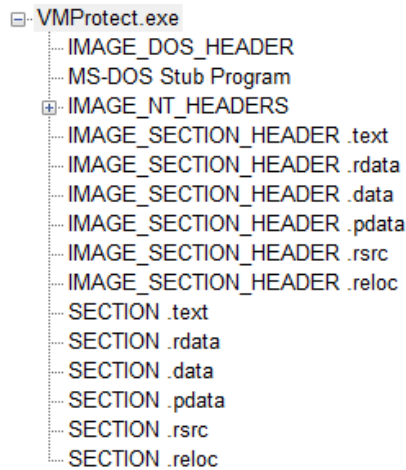


Fig. 24. VMProtect 원본 PE구조.

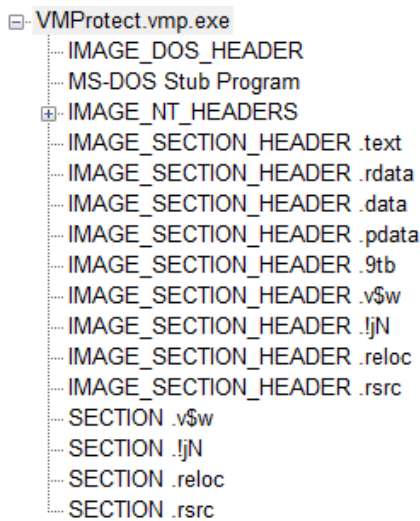


Fig. 25. VMProtect 패키징된 PE구조.

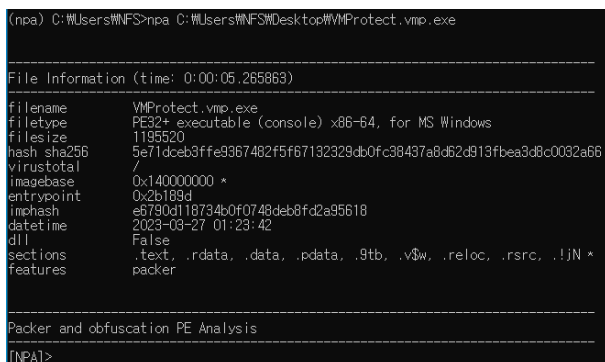


Fig. 26. 패키징파일 분석 시작.

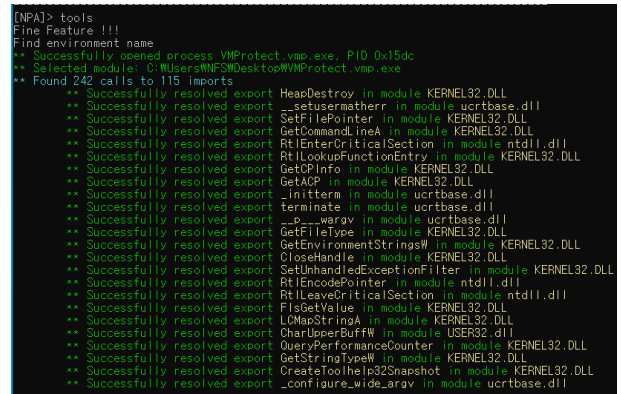


Fig. 27. VMProtect 언패킹.

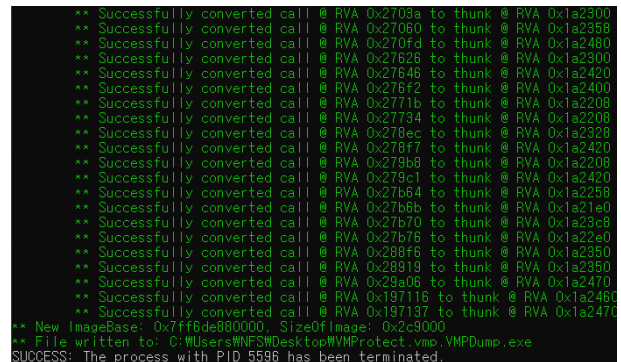


Fig. 28. VMProtect 언패킹.

테스트 코드 외에 추가로 DikeDataset에서 수집한 UPX, Themida, VMProtect로 패키징 및 난독화 된 총 475개의 파일을 대상으로 언패킹 및 역난독화 테스트를 진행하여 총 451개의 파일이 언패킹 및 역난독화된 것을 검증하였으며, 94.9%의 복원률을 보였다.

아울러, 언패킹 및 역난독화에 성공한 모든 파일은 IDA Pro와 OllyDbg를 이용하여 정적분석이 가능한 상태로 복원이 되었음을 확인하였다, 또한 PView를 이용하여 복원된 파일의 IAT가 복구된 것을 확인하였다.

IV. 결론

산업과 사회 인프라의 급속한 디지털 전환에 따라 사이버 범죄의 증가로 관련된 악성코드 및 소프트웨어에 대한 정적 분석은 기하급수적으로 증가할 것으로 예상된다. 이에 따라, 본 연구를 통해 개발된 통합 자동화 언패킹 및 역난독화

분석 시스템은 정적분석에 소요되는 인적 비용과 시간적 비용을 절감할 것으로 판단되며, 악성코드 및 소프트웨어 정적분석 역량 강화에 기여할 것으로 판단된다.

본 연구가 활용된다면 사이버 범죄 및 사고의 전체적인 프로파일링이 가능할 것으로 보여, 정보보호 기업과 수사기관 등에서 활용도가 높을 것으로 판단된다. 아울러, 분석인력 양성을 위한 교육 도구로도 활용이 가능할 것이다.

패킹과 난독화 기술은 소프트웨어의 자기보호 기능을 위해 앞으로도 지속적으로 발전할 수밖에 없는 분야로 언패킹 및 역난독화는 지속적인 연구가 수반 되어 하는 분야이다. 또한, 지속적인 연구를 통해 다양한 패커들을 추가하고, 동적 분석 시스템과의 통합 등의 연구가 진행된다면, 보다 고도화된 자동화 분석 체계 수립이 가능할 것으로 판단된다.

이후 연구로는 자동화 언패킹 및 역난독화 도구를 RPA를 활용하여 더욱 세밀하고 인적 요소가 없이 더욱 고도화된 자동화 도구가 될 것으로 판단된다.

V. 사 사

이 논문은 행정안전부 주관 국립과학수사연구원 중장기과학수사감정기법연구개발(R&D)사업의 지원을 받아 수행한 연구임(NFS2022DTB04).

VI. 참고문헌

- 김병지, 한상원, 이재광. 2021. 악성코드 특징정보(Feature)의 종류 및 시스템 적용 사례 연구. 정보보호학회지 31(3):81-87.
- 김선균, 김하진, 최미정. 2018. Anti-VM/Debugging 무력화 환경에서 악성코드 자동 언패킹 시스템 설계 및 구현. 한국통신학회 논문지 43(11):1929-1940.
- 김선균. 2018. 악성코드 분석을 위한 PE구조 언패킹 자동 시스템 설계 및 구현. 석사학위논문, 강원대학교 일반대학원.
- 이승원. 2012. 리버싱 핵심원리. 서울: 인사이트.

이재휘, 이병희, 조상현. 2019. 최신 버전의 Themida가 보이는 정규화가 어려운 API 난독화 분석방안 연구. 정보보호학회 논문지 29(6):1375-1382.

이재휘, 한재혁, 이민욱, 최재문, 백현우, 이상진. 2017. Themida의 API 난독화 분석과 복구방안 연구. 정보보호학회 논문지 27(1):67-77.

ASEC. 최신 닷넷 패커의 종류 및 국내 유포 동향. Available: <https://asec.ahnlab.com/ko/44066/#1>

Bu S, Jeong UA, Koh J. 2022. Robotic process automation: A new enabler for digital transformation and operational excellence. Bus. Commun. Res. Pract. 5(1):29-35.

GitHub. DikeDataset. Available: <https://github.com/iosifache/DikeDataset>

GitHub. Peframe. Available: <https://github.com/digitalsleuth/peframe>

GitHub. Unlicense. Available: <https://github.com/ergrelet/unlicense>

GitHub. UPX. Available: <https://github.com/upx/upx>

GitHub. Vmpdump. Available: <https://github.com/0xnobody/vmpdump>

IBM. 멀웨어란. Available: <https://www.ibm.com/kr-ko/topics/malware>

KISA. 2022국가정보보호백서. Available: <https://www.kisa.or.kr/20303/form?postSeq=12002&page=1#fnPostAttachDownload>

KISA. 2022년 상반기 사이버 위협 동향 보고서. Available: https://www.krcert.or.kr/data/reportView.do?bulletin_writing_sequece=67087

Muralidharan T, Cohen A, Gerson N, Nissim N. 2022. File packing from the Malware perspective: Techniques, analysis approaches, and directions for enhancements. ACM Computing Surveys 55(5):1-45.